

ESCUELA COLOMBIANA DE INGENIERÍA DESARROLLO ORIENTADO POR OBJETOS

Construcción 2026-2 — Laboratorio 1/6

OBJETIVO

Evidenciar el logro de los siguientes resultados de aprendizaje:

1. Apropiar un paquete revisando el diagrama de clases, la documentación y el código.
2. Crear y manipular un objeto. Extender y crear una clase.
3. Entender el comportamiento básico de memoria en la programación OO.
4. Investigar clases y métodos en el [API](#) de Java.
5. Utilizar el entorno de desarrollo de BlueJ.
6. Vivenciar las prácticas XP:

Planning — el proyecto se divide en iteraciones; **Coding** — todo el código de producción se programa en parejas.

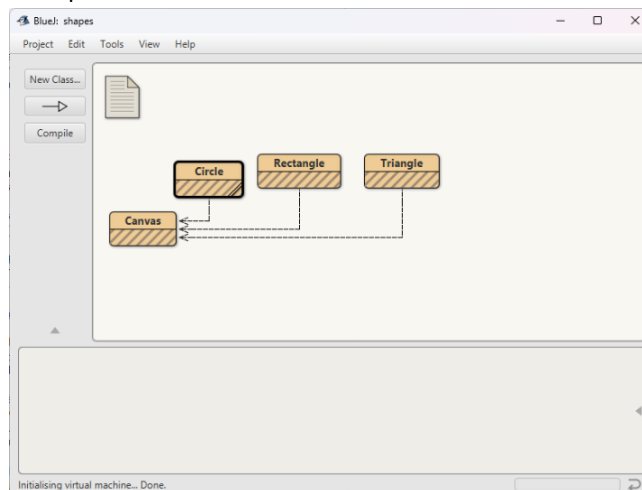
ENTREGA

Incluyan en un archivo **.zip** los archivos correspondientes al laboratorio. El nombre debe ser los apellidos de los dos miembros del equipo ordenados alfabéticamente y separados por un guion (CasallasC-DuarteJ). Publiquen el avance al final de la sesión y la versión definitiva en la fecha indicada, en los espacios correspondientes.

PARTE I. SHAPES

A Conociendo el proyecto. [En [lab01.doc](#)]

1. El proyecto [shapes](#) es una versión modificada de un recurso ofrecido por BlueJ. Para trabajar con él, bajen [shapes.zip](#) y ábralo en BlueJ¹. Capturen la pantalla.



2. El **diagrama de clases** permite visualizar las clases de un artefacto software y las relaciones entre ellas. Considerando el diagrama de clases de [shapes](#):

(a) ¿Qué clases ofrece?

Tiene 4 clases; Canvas, Circle, Rectangle y Triangle.

(b) ¿Qué relaciones existen entre ellas?

Hay relaciones de uso, en el diagrama se da que toda figura (Circle, Rectangle y Triangle) usa al Canvas.

Circle se relaciona con Canvas

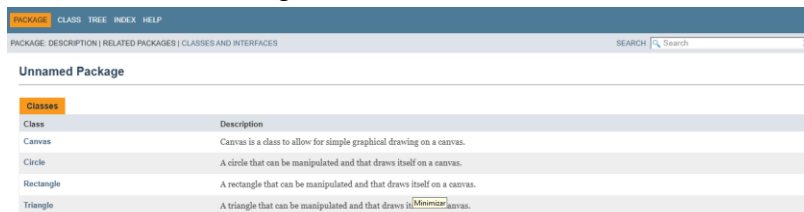
Rectangle se relaciona con Canvas

Triangle se relaciona con Canvas

3. La **documentación**² presenta las clases del proyecto y, en este caso, la especificación de sus componentes públicos. Genere la documentación. De acuerdo con la documentación:

(a) ¿Qué clases tiene el paquete [shapes](#)?

Generando la documentación se tienen las siguientes clases.



Class	Description
Canvas	Canvas is a class to allow for simple graphical drawing on a canvas.
Circle	A circle that can be manipulated and that draws itself on a canvas.
Rectangle	A rectangle that can be manipulated and that draws itself on a canvas.
Triangle	A triangle that can be manipulated and that draws itself on a canvas.

En resumen, son las mismas que se ofrecen al abrir BlueJ.

(b) ¿Qué atributos tiene la clase [Triangle](#)?

Según la documentación se tiene solamente uno llamado VERTICES.



```
Class Triangle
java.lang.Object
Triangle

public class Triangle
extends Object

A triangle that can be manipulated and that draws itself on a canvas.

Version:
1.0 (15 July 2000)

Author:
Michael Kölling and David J. Barnes
```

Field Summary

Modifier and Type	Field	Description
static int	VERTICES	

(c) ¿Cuántos métodos ofrece la clase [Triangle](#)?

Son 13 métodos, un método de clase el cual es el constructor y el resto de los métodos de objeto, sin contar los métodos heredados, especificados en la documentación.

Constructor Summary		
Constructors		
Constructor	Description	
Triangle()	Create a new triangle at default position with default color.	

Method Summary		
All Methods	Instance Methods	Concrete Methods
Modifier and Type	Method	Description
void	changeColor(String# newColor)	Change the color.
void	changeSize(int newHeight, int newWidth)	Change the size to the new size
void	makeInvisible()	Make this triangle invisible.
void	makeVisible()	Make this triangle visible.
void	moveDown()	Move the triangle a few pixels down.
void	moveHorizontal(int distance)	Move the triangle horizontally.
void	moveLeft()	Move the triangle a few pixels to the left.
void	moveRight()	Move the triangle a few pixels to the right.
void	moveUp()	Move the triangle a few pixels up.
void	moveVertical(int distance)	Move the triangle vertically.
void	slowMoveHorizontal(int distance)	Slowly move the triangle horizontally.
void	slowMoveVertical(int distance)	Slowly move the triangle vertically.

(d) ¿Qué atributos determinan el tamaño de un [Triangle](#)?

Según la documentación no hay atributos que determinen el tamaño de un triángulo, sin embargo, en el código fuente se puede evidenciar que tiene una altura (height) y una anchura (width) como atributos privados que definen el tamaño de un [Triangle](#).

(e) ¿Cuáles métodos ofrece la clase [Triangle](#) para cambiar su tamaño?

Solamente se observa un método que cambia el tamaño de la figura

Method Summary		
All Methods	Instance Methods	Concrete Methods
Modifier and Type	Method	Description
void	changeColor(String# newColor)	Change the color.
void	changeSize(int newHeight, int newWidth)	Change the size to the new size
void	makeInvisible()	Make this triangle invisible.
void	makeVisible()	Make this triangle visible.
void	moveDown()	Move the triangle a few pixels down.
void	moveHorizontal(int distance)	Move the triangle horizontally.
void	moveLeft()	Move the triangle a few pixels to the left.
void	moveRight()	Move the triangle a few pixels to the right.
void	moveUp()	Move the triangle a few pixels up.
void	moveVertical(int distance)	Move the triangle vertically.
void	slowMoveHorizontal(int distance)	Slowly move the triangle horizontally.
void	slowMoveVertical(int distance)	Slowly move the triangle vertically.

4. En el **código fuente** de cada clase está el detalle de la implementación. Revisen el código de la clase [Triangle](#). Con respecto a los atributos:

(a) ¿Cuántos atributos realmente tiene?

Tiene 7 atributos.

```
public class Triangle{  
  
    public static int VERTICES=3;  
  
    private int height;  
    private int width;  
    private int xPosition;  
    private int yPosition;  
    private String color;  
    private boolean isVisible;  
}
```

(b) ¿Quiénes pueden usar los atributos públicos?

Toda clase puede usar los atributos públicos.

Con respecto a los métodos:

(c) ¿Cuántos métodos tiene en total?

Tiene 15 métodos, los métodos privados que no aparecen en la documentación son los siguientes

```
/*  
 * Draw the triangle with current specifications on screen.  
 */  
private void draw(){  
    if(isVisible) {  
        Canvas canvas = Canvas.getCanvas();  
        int[] xpoints = { xPosition, xPosition + (width/2), xPosition - (width/2) };  
        int[] ypoints = { yPosition, yPosition + height, yPosition - height };  
        canvas.draw(this, color, new Polygon(xpoints, ypoints, 3));  
        canvas.wait(10);  
    }  
}  
  
/*  
 * Erase the triangle on screen.  
 */  
private void erase(){  
    if(isVisible) {  
        Canvas canvas = Canvas.getCanvas();  
        canvas.erase(this);  
    }  
}
```

(d) ¿Quiénes usan los métodos privados?

La única que puede usar los métodos privados es la clase que los contienen.

5. Desde el editor, consulte la documentación de [Triangle](#).

(a) Capture la pantalla.

PACKAGE

CLASS

TREE

INDEX

HELP

SUMMARY: NESTED | FIELD | CONSTR | METHODDETAIL: FIELD | CONSTR | METHOD

Class Triangle

java.lang.Object^{td}
Triangle

public class Triangle
extends Object^{td}

A triangle that can be manipulated and that draws itself on a canvas.

Version:
1.0 (15 July 2000)

Author:
Michael Kolling and David J. Barnes

Comparando la documentación con el código:

(b) ¿Qué no se ve en la documentación?

Lo que no se ve en la documentación son los métodos y atributos privados.

(c) ¿por qué debe ser así?

Por el principio de encapsulamiento que menciona que solamente deben ser manipulados los datos por medio de los comportamientos que son públicos, esto impide la manipulación directa y promueve la integridad de los datos.

6. ¿Cuál dirían es el propósito del proyecto [shapes](#)?

En lo personal el propósito de [shapes](#) es brindar una forma de pintar en un lienzo, llamado Canvas, figuras como el cuadrado, triángulo y círculo, prácticamente brindar una interfaz donde se puedan generar diferentes dibujos.

Por otro lado, se puede ver como un proyecto formativo para los estudiantes de DOPO, con el fin que aprendan los conceptos básicos de java, clases, objetos, atributos y métodos. También para familiarizarse con las herramientas utilizadas en la case.

B Manipulando objetos. [En [lab01.doc](#)]

1. Creen un objeto de cada una de las clases que lo permitan³ y háganlos visible.⁴

(a) ¿Cuántas clases tiene el proyecto?

Hay 4 clases, sin incluir que se usa la String para los colores de las figuras.

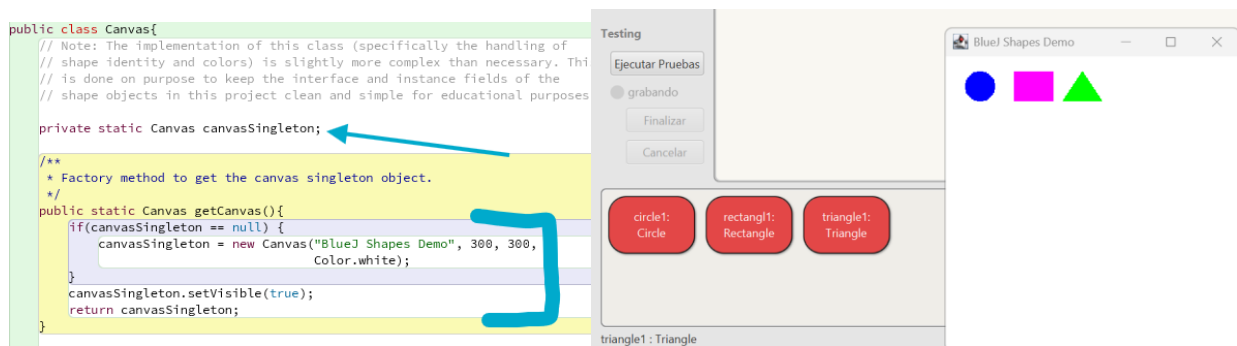
(b) ¿Cuántos objetos crearon?

Se crearon 3 figuras, por defecto al hacerlas visibles se crea también un Canvas, por lo que en realidad se crearon 4 objetos. Sin incluir que se crearon los Strings de los colores de las figuras, en ese caso serían 6 objetos.

(c) ¿Qué clase se crea de forma diferente? ¿Por qué?

La clase que se crea diferente es Canvas, de esta solamente se crea una instancia porque se usa el patrón de diseño Singleton, así como se observa en la siguiente imagen

(d) Capturen la pantalla.



2. Inspeccionen los creadores de cada una de las clases.

(a) ¿Cuál es la principal diferencia entre ellos?

En Canvas se tiene que se tiene su constructor privado y público ya que el constructor publico utiliza un método privado para crear el objeto Canvas, debido al patrón de diseño que sigue, así cómo se muestra a continuación.

```
/**
 * Create a Canvas.
 * @param title title to appear in Canvas Frame
 * @param width the desired width for the canvas
 * @param height the desired height for the canvas
 * @param bgClour the desired background colour of the canvas
 */
private Canvas(String title, int width, int height, Color bgColour){
    frame = new JFrame();
    canvas = new CanvasPane();
    frame.setContentPane(canvas);
    frame.setTitle(title);
    canvas.setPreferredSize(new Dimension(width, height));
    backgroundColour = bgColour;
    frame.pack();
    objects = new ArrayList<Object>();
    shapes = new HashMap<Object, ShapeDescription>();
}
```

Por otro lado, se observan los constructores de las demás figuras y se tiene que los únicos atributos que cambian en su instanciación es el color y las posiciones x e y.

(b) ¿Qué se busca con la clase que tiene el creador diferente?

Lo que se busca es aplicar el patrón Singleton, esto sirve entre muchas cosas para ahorrar recursos computacionales, darle sentido al diseño por lo que solamente se necesita un solo lienzo para pintar las figuras.

3. Construyan, con `shapes` sin escribir código, una propuesta de la imagen del logo de su navegador favorito.

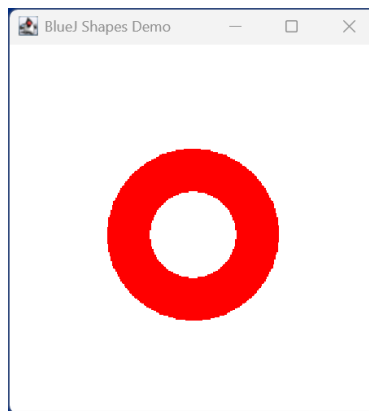
(a) ¿Cuántas y cuáles clases se necesitan?

Se piensa hacer el navegador de Opera, las clases que se necesitan es 1, el círculo.

(b) ¿Cuántos objetos se usan en total?

Las instanciaciones de esta clase van a ser dos.

(c) Capturen la pantalla.



(d) Incluyan el logo original



Nota. Tomado de *Historia del logo de Opera*, por 1000 Marcas, 2023 (<https://1000marcas.net/opera-logo/>).

C. Detallando una clase y explorando sus objetos. [En `lab01.doc`]

1. En el código de la clase `Triangle`, revise el atributo `VERTICES`.

(a) ¿Qué significa que sea `public`?

Esto significa que cualquier otra clase en cualquier parte del programa puede ver, leer y modificar ese valor de manera directa, sin restricciones de paquetes ni la necesidad de métodos adicionales.

(b) ¿Qué significa que sea `static`?

Que se crea una instancia al ejecutar el código y solamente se crea una durante la ejecución de todo el programa con el mismo valor y dirección en memoria.

(c) ¿Debería ser `final`? ¿Por qué?

Por el encapsulamiento, este principio se debe quitar el acceso a modificar datos del producto que se quiera entregar.

(d) Realicen los cambios necesarios.

```
0 public class Triangle extends StraightSided {  
1  
2     public static final int VERTICES=3;
```

2. En el código de la clase `Triangle` revisen el detalle del tipo del atributo `height`.

(a) ¿Qué se está indicando al decir que es `int`?

Se indica que es un atributo con dato primitivo que sirve para almacenar solo números enteros que ocupen 32 bits

(b) Si fuera `byte`, ¿cuál sería el área más grande posible?

Ya que un byte son 8 bits, el número más grande es 127 y ese sería el valor máximo del área.

(c) y ¿si fuera `long`?

Log ocupa 64 bits, su valor más grande es: 9,223,372,036,854,775,807 (operación por Gemini IA).

(d) ¿qué restricción adicional debería tener este atributo?

La restricción adicional que debe tener `height` es que no puede ser más grande que el mismo `Canvas` que tiene una altura de 300.

(e) Refactoricen el código considerando.

```
public void changeSize(int newHeight, int newWidth) {  
    erase();  
    if (newHeight < 300) {  
        height = newHeight;  
    }  
    width = newWidth;  
    draw();  
}
```


3. Si creamos 100 triángulos,

(a) ¿cuántos **VERTICES** y cuántos **height** tendríamos?

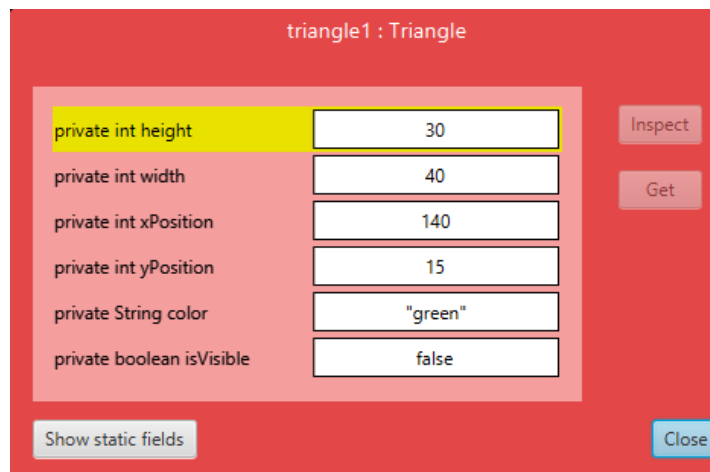
Dado a que vértices es **STATIC**, entonces solo se genera una instancia en la clase de la cual todos los objetos hacen referencia con los vectores, diferencia de **height** que es una variable, por lo tanto, cada objeto tiene su propia instancia del atributo.

(b) Justifique.

En java, los atributos que se establecen como **STATIC** pertenecen a un área de la memoria llamada *metaspace* o *permanent generacion* y obliga que solo exista una copia de esa instancia del atributo.

4. Inspeccionen el **estado** del objeto **:Triangle¹**.

(a) Capturen la pantalla.



(b) ¿Cuál es el valor de **height** inicial?

El valor inicial de *height* es 30.

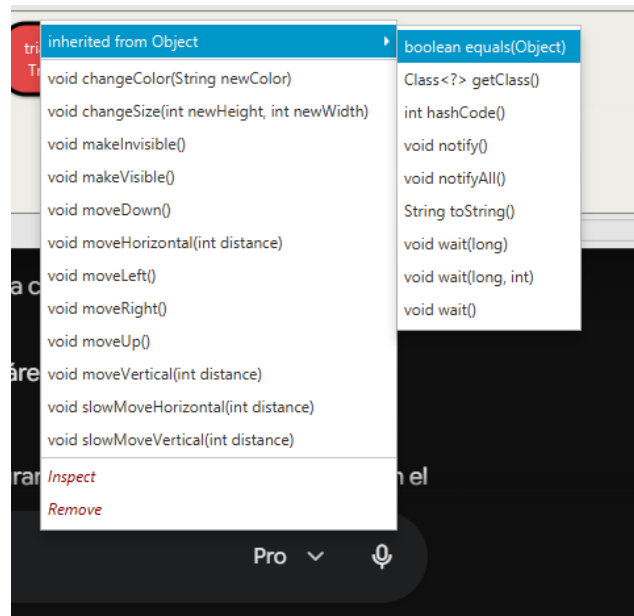
(c) ¿Cuál es el valor de **VERTICES**?

El valor de vértices es 3 ya que es constante para todos los objetos instanciados de la clase triángulo.

¹ Clic derecho sobre el objeto y comando *Inspect* 6
Clic derecho sobre el objeto

5. Inspeccionen el **comportamiento** que ofrece el objeto `:Triangle`⁶.

(a) Capturen la pantalla.



(b) ¿Por qué no aparecen todos los métodos que están en el código?

Porque no todos los métodos son públicos `draw()` y `erase()` son privados.

D. Analizando y escribiendo código. [En lab01.doc]

```
Circle face;
Circle eye;
Triangle mouthUp;
Triangle mouthDown;
//1
face = new Circle();
face.changeColor("yellow");
face.changeSize(100);
//2
eye = new Circle();
eye.changeColor("black");
eye.changeSize(15);
eye.moveVertical(10);
eye.moveHorizontal(35);
eye.makeVisible();
//3

mouthUp = new Triangle();
mouthUp.changeColor("white");
mouthUp.changeSize(30, 90);
mouthUp.moveHorizontal(-20);
mouthUp.moveVertical(15);
mouthUp.makeVisible();
//4
mouthDown = new Triangle();
mouthDown.changeColor("white");
mouthDown.changeSize(-30, 90);
mouthDown.moveHorizontal(-20);
mouthDown.moveVertical(75);
mouthDown.makeVisible();
//5
Circle food=eye;
food.changeColor("red");
food.moveHorizontal(100);
food.moveVertical(25);
food.makeVisible();
//6
```

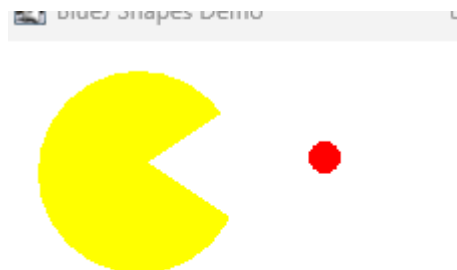
1. Lean el código anterior.

(a) ¿Cuál creen que es la figura resultante?

Se cree que es así la figura resultante: sería una especie de cara con tojos y triángulos arriba.

(b) Píntenla.

En este caso el único cambio realizado es dejar que los objetos se vieran, porque en realidad no se debería ver nada ya que en el código no se especifica que se vean los objetos excepto el ojo (el punto rojo).



2. Para cada punto señalado:

(a) ¿cuántas variables existen?

Por circulo son 5 variables y son 2 círculos y por triangulo son 6 variables y son 2 triángulos, por tanto, son 22 variables.

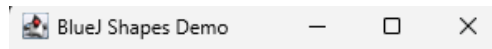
(b) ¿cuántos objetos existen?

4 objetos

(c) ¿cuántos objetos se ven? (no cuenten ni los objetos `String` ni el objeto `Canvas`) (3 respuestas en cada punto)

Se ve solo un objeto

3. Habiliten la ventana de código en línea¹, escriban el código. (a) Capturen la pantalla.



4. Compare la figura pintada en 1. con la figura capturada en 3.: (a) ¿son iguales? (b) ¿por qué?

Si son iguales si quitamos los supuestos de los objetos visibles

¹ Menú: [View](#) → [Show Code Pad](#)

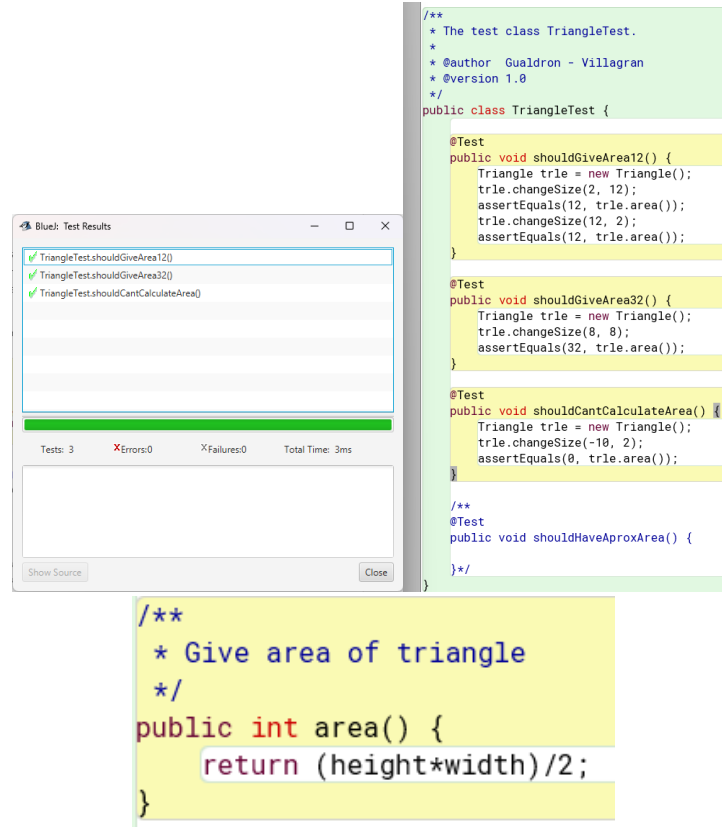
5. (a) ¿Qué errores hacen que la figura no quede bien pintada? (b) ¿Qué figura es?

No se ejecutan los métodos para que se venan las clases y la variable comida comparte la misma dirección en memoria que eye, entonces se sobre escriben

E Extendiendo una clase. [Triangle](#). [En [lab01.doc](#)]

1. Desarrollen en [Triangle](#) el método `area()` (retorna el área). ¡Pruébenlo! Capturen una pantalla.

Se ve en la siguiente imagen



2. Desarrollen en [Triangle](#) el método `equilateral()` (lo convierte en un triángulo equilátero de aproximadamente la misma área) ¡Pruébenlo! Capturen dos pantallas.

Código de método

```
/**
 * Convert this triangle into an equilateral triangle
 */
public void equilateral() {
    erase();
    /* Según  $area = s^2 * (\sqrt{3}) / 4$  <- Formula de area del triangulo equilatero
     *  $s = \sqrt{4 * area / \sqrt{3}}$ 
     */
    double s = Math.sqrt((4 * area()) / Math.sqrt(3)); // Lado de triang. equilatero

    width = (int) s;
    height = (int) (s * (Math.sqrt(3) / 2)); /* Propiedad de la altura
     de una triangulo equilatero:  $s * \sqrt{3} / 2$  */

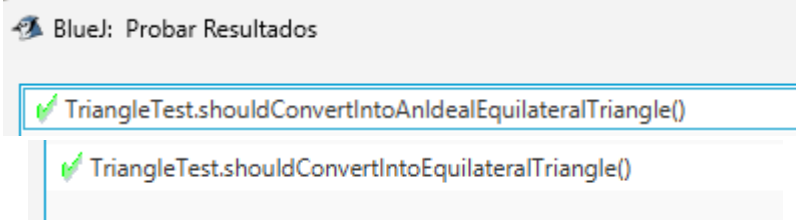
    draw();
}
```

Pruebas

```
@Test
public void shouldConvertIntoAnIdealEquilateralTriangle() {
    Triangle trle = new Triangle();
    trle.changeSize(140, 230);
    trle.equilateral(); /* Tiene que cumplir que sean areas aproximadamente cercanas al area ideal de un triangulo equilatero.
     Esto por el truncamiento */
    int areaTriangle = trle.area(), proofAreaEquilateral = (int) ((Math.sqrt(3)/4)*Math.pow(trle.getWidth(), 2));
    double delta = proofAreaEquilateral*0.01 /* <- tolerancia */;
    assertEquals(areaTriangle, proofAreaEquilateral, delta); /* delta es el error que puede
     cometer el calculo de proofAreaEquilateral */
}

@Test
public void shouldConvertIntoEquilateralTriangle() {
    Triangle trle = new Triangle();
    trle.changeSize(9401, 2307); // Si se cambia esto llega hasta un límite, después falla
    int areaBefore = trle.area();
    trle.equilateral(); /* Tiene que cumplir que sean areas aproximadamente
     cercanas al area antes de convertirlo a equilatero */
    int areaTriangleEq = trle.area();
    double delta = areaTriangleEq*0.02 /* <- tolerancia */;
    assertEquals(areaTriangleEq, areaBefore, delta); // delta es el error que puede cometer el calculo del area
}
```

Ejecución de pruebas



3. Desarrollen en `Triangle` el método `walk(times: int)` (va moviéndose cayendo hacia la derecha (positivo) o izquierda (negativo) `abs(times)` veces). ¡Pruébenlo! Capturen tres pantallas.

Código

```
/**
 * Make to move triangle # times,
 * triangle moves 10 units depending on if times is positive this triangle
 * shifts to the right, but if times is negative figure shifts to the left
 * @param times times are the number of iterations (10 units per iteration)
 * that this figure moves based on if this number is negative or positive
 */
public void walk(int times) {
    slowMoveHorizontal(times*10);
}
```

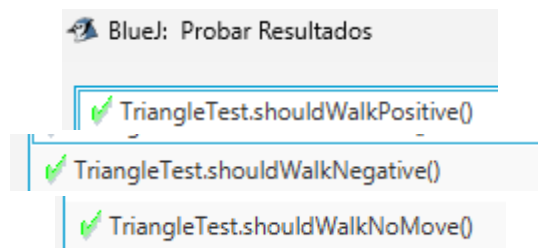
Código de pruebas

```
@Test
public void shouldWalkPositive() {
    int xPosBefore = trle.getXPosition();
    trle.walk(5);
    int movement = 5*10;
    int xPosNow = trle.getXPosition();
    assertEquals(xPosNow, xPosBefore+movement);
}

@Test
public void shouldWalkNegative() {
    int xPosBefore = trle.getXPosition();
    trle.walk(-7);
    int movement = (-7)*10;
    int xPosNow = trle.getXPosition();
    assertEquals(xPosNow, xPosBefore+movement);
}

@Test
public void shouldWalkNoMove() {
    int xPosBefore = trle.getXPosition();
    trle.walk(0);
    int movement = 0;
    int xPosNow = trle.getXPosition();
    assertEquals(xPosNow, xPosBefore+movement);
}
```

Ejecución de pruebas



4. Desarrollen en `Triangle` un nuevo creador que permita crearlo dado su color, su ancho y su alto. ¡Pruébenlo! Capturen una pantalla.

Código de método:

```
/**
 * Create a new triangle given a color, width and height
 */
public Triangle(String color, int height, int width) {
    super(140, 15, color);
    this.height = height;
    this.width = width;
    rotationAngle = 0.0;
}
```

Pruebas de constructor:

```
@Test
public void shouldCreateASpecificTriangle() {
    StraightSided specificTriangle = new Triangle("magenta", 32, 78);
    assertEquals("magenta", specificTriangle.getColor());
    assertEquals(32, specificTriangle.getHeight());
    assertEquals(78, specificTriangle.getWidth());
}
```

A la ejecución de la prueba:

✓ TriangleTest.shouldCreateASpecificTriangle()

5. Propongan un nuevo método para esta clase. Desarrollen y prueben el método.

El nuevo método va a ser que se voltee el triángulo dado, el código de este es el siguiente

```
/**
 * Flips triangle vertically. Inspiración: Gemini IA
 */
public void flip() {
    erase();
    height = -height;
    draw();
}
```

La prueba unitaria es:

```
@Test
public void shouldFlipTringle() {
    int alturaInicial = trle.getHeight();
    int yPosBaseInicial = trle.getYPosition() + alturaInicial;
    trle.flip();
    int yPosBaseFinal = trle.getYPosition() - alturaInicial;
    assertEquals(-alturaInicial, trle.getHeight());
    assertEquals(2*alturaInicial, Math.abs(yPosBaseInicial-yPosBaseFinal));
    /*Si se volteó el triangulo la yPosicion actual menos la yPosicion final debe ser
    el doble de la altura*/
}
```

Cuando se ejecuta resulta como válida así como se muestra a continuación

✔ TriangleTest.shouldFlipTringle()

6. Generen nuevamente la documentación y revisen la información de estos nuevos métodos. Capturen la pantalla.

Unnamed Package

Classes	
Class	Description
Canvas	Canvas is a class to allow for simple graphical drawing on a canvas.
Circle	A circle that can be manipulated and that draws itself on a canvas.
Figure	Class Figure, each figure of shapes is Figure
Rectangle	A rectangle that can be manipulated and that draws itself on a canvas.
Robot	This is the robot of maze.
StraightSided	StraightSided are the figures that has a height and a width.
Triangle	A triangle that can be manipulated and that draws itself on a canvas.
TriangleTest	The test class TriangleTest.

Ahora la documentación de los métodos son los siguientes

Method Summary

All Methods	Instance Methods	Concrete Methods
Modifier and Type	Method	Description
int	<code>area()</code>	Give area of triangle
protected void	<code>draw()</code>	Draw the shape on canvas
void	<code>equilateral()</code>	Convert this triangle into an equilateral triangle
void	<code>flip()</code>	Flips triangle vertically.
void	<code>rotate(double angle)</code>	This able to rotate the triangle a determinated angle
void	<code>walk(int times)</code>	Make to move triangle # times, triangle moves 10 units depending on if times is positive this triangle shifts to the righth, but if times is negative figure shifts to the left

PARTE II. De Python a Java

En este punto vamos a usar y evaluar dos recursos de apoyo para la transición de Python a Java. Realicen la evaluación en las encuestas preparadas con ese objetivo:

- 1. El video.
- 2. Los prompts.

Por parte de Samuel ya se hizo la evaluación.

PARTE III. RobotMaze

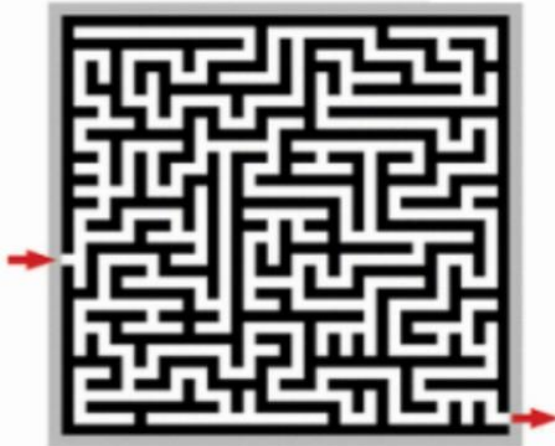
El objetivo de este punto es desarrollar una mini-aplicación para una versión simplificada del juego RobotMaze. Este juego está inspirado en Karel, un emulador de un robot, distribuido como software educativo de código abierto, creado Richard E. Pattis en su libro Karel The Robot:

A Gentle Introduction to the Art of Programming. [VINCULO](#)

El juego se desarrolla en un laberinto con una entrada, una salida y un conjunto de paredes. El robot inicia en la entrada y el usuario controla sus movimientos mediante comandos. El objetivo del juego es guiar al robot hasta la salida consumiendo la menor cantidad posible de vidas. El jugador indica el tamaño del juego. La entrada se crea en una posición aleatoria y la salida en una posición aleatoria de la cara opuesta.

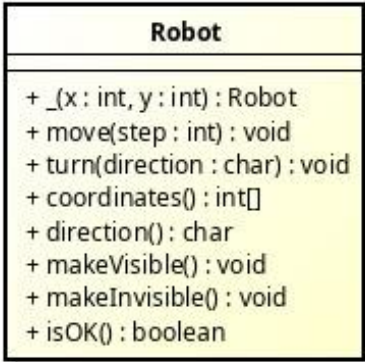
- ☐ El jugador coloca las paredes en el laberinto.
 - ☐ Cuando el jugador decide iniciar el juego, el robot se ubica en la entrada. A partir de este momento puede mover el robot que tiene 10 puntos de vida.
 - ☐ El jugador puede pedirle al robot que gire o que avance una casilla en la dirección actual
 - ☐ El robot puede girar en las cuatro direcciones (North, South, East, West)
 - ☐ Si el robot choca contra una pared o los bordes del laberinto, pierde 1 punto de vida.
- Si el robot se queda sin vida, no puede realizar ningún movimiento.

ROBOT MAZE 



A Creando una nueva clase. Usando el paquete `shapes` [En `lab01.doc`]

En este punto vamos a crear la clase `Robot` del juego. El diseño gráfico lo definen ustedes.

	Ciclo 1: - <code>_()</code> - o <code>coordinates()</code> - o <code>direction()</code> Ciclo 2: - o <code>move()</code> - o <code>turn()</code> - o <code>isOK()</code> Ciclo 3: - o <code>makeVisible()</code> - o <code>makeInvisible()</code> Notas: Puede mirar a cuatro posibles direcciones: N, S, E, W. Se crea mirando al norte. <code>coordinates</code> retorna la posición {x,y} En <code>move</code>, el parámetro puede ser negativo <code>isOK</code> retorna falso si no se pudo hacer el último movimiento
---	--

1. Inicie la construcción únicamente con los **atributos**. Justifique su selección. Agregue pantallazo con los atributos.

```
protected int xPosition;
protected int yPosition;
protected boolean isVisible;
protected ArrayList<Figure> robotFigure;

protected int live;
protected boolean ok;
protected char direction;

public static final int HEAD = 0;
public static final int BODY = 1;
public static final int MOUTH = 2;
public static final int ANTENNA = 3;
public static final int LEFT_EYE = 4;
public static final int RIGHT_EYE = 5;
public static final int NEEDLE = 6;
public static final int NUM_FIGURES = 6;
```

- `x/yPosition` es la posición de nuestro robot.
- `isVisible` permite conocer de la clase si el robot está visible en el *canvas* o no.
- `robotFigure` permite por medio del polimorfismo agrupar toda la figura del robot la cual es 3 cuadrados, 2 círculos y un triángulo.
- `ok` permite conocer el estado del robot, si por ejemplo este ya pasó a la meta o si está muerto, retorna falso si no se pudo ejecutar el próximo movimiento.
- `direction` lo que indica es la dirección del robot para moverse que está en inglés su dirección las cuales son North, South, East, West se observa que como es un *char* lo que indica es que si es 'n' → North, si es 's' → South, si es 'e' → East y si es 'w' → West.
- Las constantes indican los índices en los que se guardan las figuras en el `ArrayList<Figure>`.
- `Live` indica la vida que tiene el robot se pierde vida en cada colisión detectada.

2. Desarrollen la clase considerando los 3 mini-ciclos. No olviden la documentación. Al final de cada mini-ciclo realicen dos pruebas indicando su propósito. Capturen las pantallas relevantes.

Ciclo 1:

- `_()`
- `coordinates()`
- `Direction()`

```
//ciclo 1
/**
 * Constructor to able create a robot
 * with specific coordenades of canva.
 * @param x x is the x robot's position on the canvas.
 * @param y y is the y robot's position on the canvas.
 */
public Robot(int x, int y) {
    this.xPosition = x;
    this.yPosition = y;

    this.robotFigure = new ArrayList<>();
    this.isVisible = true;
    this.direction = 'e';
    this.live = 10;

    makeRobotShapes();
    modifyRobotShapes(); /* Add instructions at methods
                           to make more simplified this method of class*/
}

/** Give the x and y coordenades of robot.
 * @return return robot's coordinates (x,y)
 */
public int[] coordinates() {
    int[] coordinatesXY;
    coordinatesXY = new int[2];
    coordinatesXY[0] = xPosition;
    coordinatesXY[1] = yPosition;
    return coordinatesXY;
}

/**
 * Give the direction of robot.
 * @return return attribute direction of robot.
 */
public char direction() {
    return this.direction;
}
```

Ciclo 2:

- move()
- turn()
- isOK()

```
/**
 * Move the figure according to the direction
 * @param step step are the times that robot moves.
 */
public void move(int step) {
    int dx = 0, dy = 0;
    nextNumStep = step;

    switch (this.direction) {
        case 'w':
            // izquierda
            if (this.canMove) {
                dx = -1;
            }
            break;
        case 's':
            // abajo
            if (this.canMove) {
                dy = 1;
            }
            break;
        case 'e':
            // derecha
            if (this.canMove) {
                dx = 1;
            }
            break;
        case 'n':
            // arriba
            if (this.canMove) {
                dy = -1;
            }
            break;
    }
    moveSlow(dx, dy, step);
}

/**
 * change direction
 * @param direction direction can be 'n', 'w', 'e' o 's'.
 */
public void turn(char direction){
    ((Triangle)robotFigure.get(NEDLE)).erase();
    if (direction == 'n') {
        ((Triangle)robotFigure.get(NEDLE)).rotate(0);
    } else if (direction == 'w') {
        ((Triangle)robotFigure.get(NEDLE)).rotate(-90);
    } else if (direction == 'e') {
        ((Triangle)robotFigure.get(NEDLE)).rotate(90);
    } else if (direction == 's') {
        ((Triangle)robotFigure.get(NEDLE)).rotate(180);
    }

    ((Triangle)robotFigure.get(NEDLE)).draw();
    this.direction = direction;
}
```

```

/**
 * Indicatns if the robot can move (if this have live).
 */
public boolean isOK(){
    //pendiente de laberinto
    this.ok = this.live > 0;
    if (this.ok) {
        this.canMove = true;
    } else {
        canMove = false;
    }
    return this.ok;
}

```

Ciclo 3:

- makeVisible()
- makeInvisible()

```

/**
 * Hide the robot of canvas
 */
public void makeInvisible() {
    for (Figure f : robotFigure) {
        f.makeInvisible();
    }
    this.isVisible = false;
}

/**
 * Make visible robot on canvas
 */
public void makeVisible(){
    for (Figure f : robotFigure) {
        f.makeVisible();
    }
    this.isVisible = true;
}

```

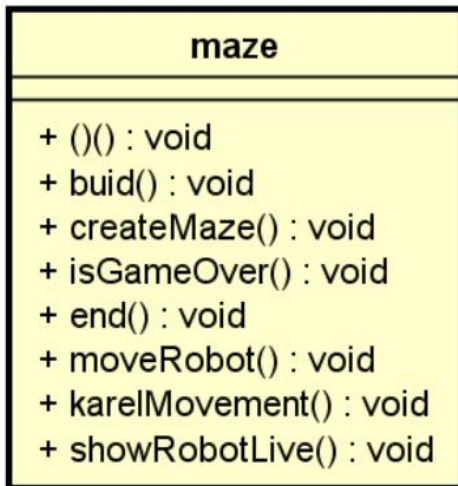
B Definiendo y creando una nueva clase. RobotMaze. [En lab01.doc]

En este punto vamos a desarrollar la *mini-aplicación* para RobotMaze.

Requisitos funcionales	Requisitos de interfaz
• Crear el laberinto indicando su tamaño. (Ubica la entrada y la salida) • Adicionar paredes al laberinto. (Únicamente antes de iniciar el juego) Iniciar el juego. • Hacer los movimientos deseados • Consultar la vida disponible. • Indicar cuando se cumplen las condiciones de fin del juego: el robot alcanza la salida o el robot se queda sin vida. • Terminar el juego, cuando se desee.	• La entrada y la salida deben diferenciarse. • La dirección a la que mira el robot debe ser evidente. • Las paredes con las que se golpea el robot adquieren un color diferente. • El robot debe lucir diferente si se quedó sin vida. • En caso de que no sea posible realizar una de las acciones, debe generar un mensaje de error. • Para los mensajes deben usar JOptionPane.

1. Diseñen la clase, es decir, definan los métodos que debe ofrecer.

Se ofrecerán dos clases para construir, el laberinto, Room y el laberinto que utiliza Room para generarse.

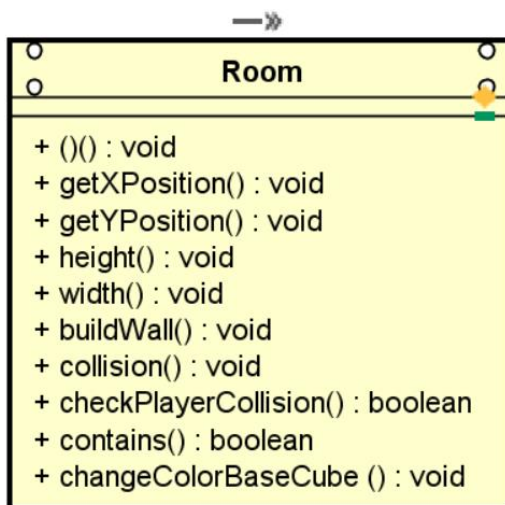


Ciclo 1:

```
--()
createMaze
Buid ()
MoveRobot()
KarelMovement()
```

Ciclo 2:

```
ShowRobotLive()
isGameOver()
End()
```



Ciclo 1:

```
--()
xPosition()
yPosition()
BuilWall ()
Height ()
Width ()
Ciclo 2:
Collision ()
CheckPlayerCollision ()
Contains ()
ChangeColorBaseCube ()
```

Planifiquen la construcción considerando algunos mini-ciclos.

2. Implementen la clase. Al final de cada mini-ciclo realicen una prueba indicando su propósito. Capturen las pantallas relevantes.

```
public class Maze {  
    private Room[][] matrix;  
    private Room roomEntry;  
    private Room roomExit;  
  
    private Robot karel;  
    private Canvas maze;  
    public static final int HEIGHT_ENTRY_ROOM = 100;  
    public static final int HEIGHT_ROBOT = 40;  
  
    /**  
     *  
     * @return return:  
     */  
    public Maze(int size) {  
        createMaze(size);  
        int gr = roomEntry.height() / 6, freeSpace = 5 * gr;  
        int startX = roomEntry.height() / 2, startY = HEIGHT_ENTRY_ROOM + gr + (freeSpace - HEIGHT_ROBOT) / 2;  
        karel = new Robot(startX, startY);  
        karel.makeVisible();  
    }  
}
```

```
/**  
 *  
 * @return return:  
 */  
public void buid(int x, int y, char direction) {  
    if (karel.getXPosition() == roomEntry.height()/2 && karel.getYPosition() == roomEntry.height()+(roomEntry.height()/2)) {  
        matrix[x][y].buildWall(direction);  
    }  
}
```

```
/**  
 *  
 * @return return:  
 */  
private void isGameOver() {  
    int xRobot = karel.getXPosition(), yRobot = karel.getYPosition();  
    if (roomExit.contains(xRobot, yRobot)) {  
        JOptionPane.showMessageDialog(  
            null,  
            "WIN",  
            "Game Over",  
            JOptionPane.INFORMATION_MESSAGE  
        );  
    } else if (!karel.isOK()) {  
        JOptionPane.showMessageDialog(null, "GAME OVER - Has perdido");//con ayuda de Gemini IA  
        end();  
    }  
}
```

```

/**
 *
 * @return return:
 */
public void moveRobot(char direction, int step) {
    karel.turn(direction);
    Room[] roomsAroundKarel = new Room[4];
    int xPosKarel = karel.getXPosition(), yPosKarel = karel.getYPosition();

    karelMovement(step);
    isGameOver();
}

public void showRobotLive() {
    karel.live();
}

private void karelMovement(int step) {
    for (int i = 0; i < step*HEIGHT_ENTRY_ROOM; i++) {
        karel.checkCollision(matrix);
        karel.checkCollision(roomEntry);
        karel.checkCollision(roomExit);
        karel.move(1);
        if (!karel.canMove()) {
            karel.isOK();
            break;
        }
        karel.isOK();
    }
}
}

```

```

/**
 *
 * @return return:
 */
public void end() {
    int size = Integer.parseInt( JOptionPane.showInputDialog("tamaño del laberinto: ") );
    maze.close();
    matrix = new Room[size][size];
    maze = null;
    createMaze(size);
    karel = null;
    karel = new Robot(roomEntry.height()/2, roomEntry.height()+(roomEntry.height()/2));
    karel.makeVisible();
}

```

Se hicieron pruebas de aceptación más que unitarias para Maze

```
public class MazeTest
{
    /**
     * Tests the robot's ability to successfully move towards or pass the goal.
     * Initializes a maze of size 5 and moves the robot East by 2 spaces,
     * followed by South by 2 spaces.
     */
    @Test
    public void shouldPassGoal() {
        Maze game = new Maze(5);
        game.moveRobot('e',2);
        game.moveRobot('s',2);
    }

    /**
     * Tests the creation of multiple mazes and the building of elements
     * (such as walls or boundaries) in different directions.
     * Verifies the {@code build} method for West ('w'), East ('e'),
     * South ('s'), and North ('n').
     */
    @Test
    public void createMaze() {
        Maze game = new Maze(1);
        game.createMaze(5);
        game.createMaze(2);
        game.createMaze(6);
        game.build(1,1,'w');
        game.build(1,1,'e');
        game.build(1,1,'s');
        game.build(1,1,'n');
    }

    /**
     * Tests a "game over" scenario.
     * Initializes a small maze (size 1) and attempts to move the robot
     * East by 2 spaces, which should theoretically trigger a game over state
     * due to moving out of bounds or hitting a barrier.
     */
    @Test
    public void gameOver() {
        Maze game = new Maze(1);
        game.moveRobot('e',2);
    }

    /**
     * Tests the termination or finalization sequence of the game.
     * Calls the {@code end()} method to ensure the game shuts down
     * or cleans up its state correctly.
     */
    @Test
    public void end() {
        Maze game = new Maze(1);
        game.end();
    }

    @Test
    public void shouldCollideRobot() {
        Maze game = new Maze(1);
        game.moveRobot('e', 1);
        game.moveRobot('n', 1);
        game.moveRobot('s', 1);
    }
}
```

Las pruebas pasan porque no se comprueba unitariamente nada, por esto entonces se puede decir que es más visual las pruebas que de comprobar la lógica interna del código.


```

/**
 * create Room
 * @param x x is the x room's position
 * @param y y is the y room's position
 */
public Room(int x, int y) {
    // Hecho con Gemini IA
    xPosition = x;
    yPosition = y;
    //
    createRoom(x, y);
}

```

```

/**
 * make visible walls
 * @return return none
 */
public void buildWall(char visible){
    switch (visible) {
        case 'w':
            leftCube.buildWall();
            break;
        case 'n':
            aboveCube.buildWall();
            break;
        case 'e':
            rightCube.buildWall();
            break;
        case 's':
            belowCube.buildWall();
            break;
    }
}

```

```

/**
 * changes color in collision the walls
 * @return return none
 */
public void collision(char collision){
    switch (collision) {
        case 'w':
            leftCube.changeColor("red");
            break;
        case 'n':
            aboveCube.changeColor("red");
            break;
        case 'e':
            rightCube.changeColor("red");
            break;
        case 's':
            belowCube.changeColor("red");
            break;
    }
}

```

```

/**
 * Check for an object colliding with the walls of this room
 * @return return none
 */
public boolean checkRobotCollision(int pX, int pY, int pWidth, int pHeight) {
    int w = width();
    int h = height();
    int grosorParedX = w / 6;
    int grosorParedY = h / 6;

    // Comprobar colisión con la pared Izquierda (w)
    if (leftCube.isVisible() &&
        pX < xPosition + grosorParedX && pX + pWidth > xPosition &&
        pY < yPosition + h && pY + pHeight > yPosition) {
        collision('w');
        return true;
    }

    // Comprobar colisión con la pared Derecha (e)
    if (rightCube.isVisible() &&
        pX < xPosition + w && pX + pWidth > xPosition + (5 * grosorParedX) &&
        pY < yPosition + h && pY + pHeight > yPosition) {
        collision('e');
        return true;
    }

    // Comprobar colisión con la pared Superior (n)
    if (aboveCube.isVisible() &&
        pX < xPosition + w && pX + pWidth > xPosition &&
        pY < yPosition + grosorParedY && pY + pHeight > yPosition) {
        collision('n');
        return true;
    }

    // Comprobar colisión con la pared Inferior (s)
    if (belowCube.isVisible() &&
        pX < xPosition + w && pX + pWidth > xPosition &&
        pY < yPosition + h && pY + pHeight > yPosition + (5 * grosorParedY)) {
        collision('s');
        return true;
    }

    return false;
}

public boolean contains(int x, int y) {
    return x >= xPosition && y >= yPosition &&
        x <= xPosition+width() && y <= yPosition+height();
}

public void changeColorBaseCube (String color) {
    baseCube.changeColor(color);
}

```

```

public class RoomTest {
    private Room room;

    /**
     * Sets up the test environment before each test method runs.
     * Initializes a new room at coordinates (0,0) and hides the canvas
     * to prevent visual rendering during the automated tests.
     */
    @BeforeEach
    public void setUp() {
        room = new Room(0, 0);
        Canvas.getCanvas().setVisible(false);
    }

    /**
     * Tests that no collisions are detected in any direction (West, East, North, South)
     * when the room has no walls built.
     */
    @Test
    public void shouldNotCollideWhenNoWallIsBuilt() {
        assertFalse(room.checkRobotCollision(0, 50, 5, 5));
        assertFalse(room.checkRobotCollision(85, 50, 5, 5));
        assertFalse(room.checkRobotCollision(40, 0, 10, 10));
        assertFalse(room.checkRobotCollision(40, 85, 10, 10));
    }

    /**
     * Tests that a collision is only detected on a specific wall that has been built.
     * In this case, only the North wall is built, so collisions should only
     * occur on the North boundary, leaving the South, West, and East boundaries open.
     */
    @Test
    public void shouldCollideOnlyWithTheBuiltWall() {
        room.buildWall('n');

        assertTrue(room.checkRobotCollision(40, 0, 10, 10));
        assertFalse(room.checkRobotCollision(40, 85, 10, 10));
        assertFalse(room.checkRobotCollision(0, 50, 5, 5));
        assertFalse(room.checkRobotCollision(85, 50, 5, 5));
    }

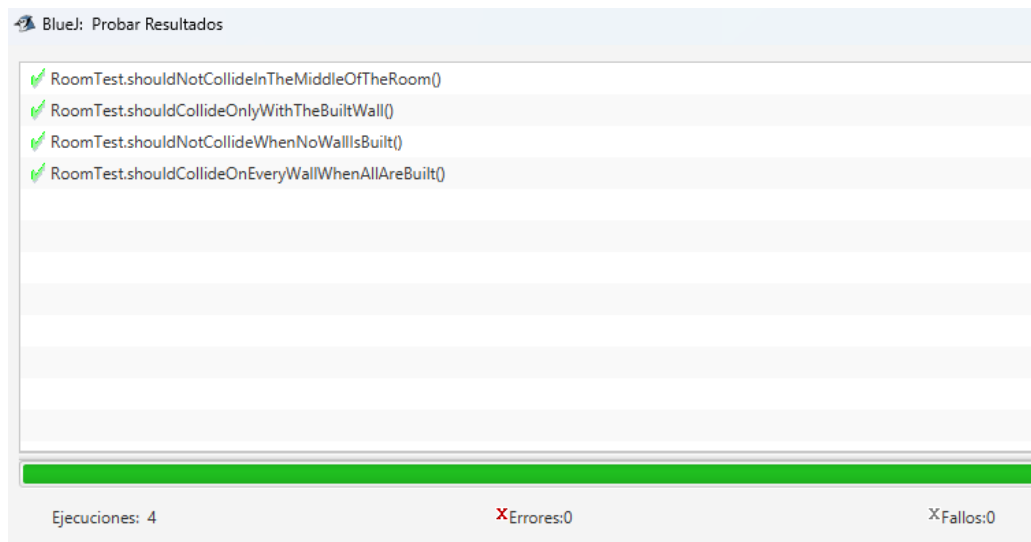
    /**
     * Tests that collisions are successfully detected on all boundaries
     * when all four walls (North, South, East, West) are built.
     */
    @Test
    public void shouldCollideOnEveryWallWhenAllAreBuilt() {
        room.buildWall('n');
        room.buildWall('s');
        room.buildWall('e');
        room.buildWall('w');

        assertTrue(room.checkRobotCollision(40, 0, 10, 10));
        assertTrue(room.checkRobotCollision(40, 85, 10, 10));
        assertTrue(room.checkRobotCollision(85, 50, 5, 5));
        assertTrue(room.checkRobotCollision(0, 50, 5, 5));
    }

    /**
     * Tests that even with all walls built, a player located safely in the middle
     * of the room (away from all edges) does not trigger any collisions.
     */
    @Test
    public void shouldNotCollideInTheMiddleOfTheRoom() {
        room.buildWall('n');
        room.buildWall('s');
        room.buildWall('e');
        room.buildWall('w');

        assertFalse(room.checkRobotCollision(40, 40, 10, 10));
    }
}

```



3. Indiquen las extensiones necesarias para reutilizar la clase `Robot` y el paquete `shapes`. Expliquen.

Si por extensiones se refiere a las librerías de robot son las siguientes

```
import java.util.ArrayList;
import javax.swing.JOptionPane;

/**
 * This is the robot of maze.
 * @author Gualdron - Villagran
 * @version 1.0
 */
public class Robot {
```


Estas librerías se deben a que Robot usa un atributo que es una colección el cual se llama *robotFigure*, este atributo es un *ArrayList* de *Figure*, por ello hay que importar esta librería.

Por parte de la librería de *swing* que importa la clase *JOptionPane* es porque en algunos casos se usa para mostrar mensajes al usuario, así como se muestra a continuación

```
public void live() {
    String message = "Karel's live is " + live;
    JOptionPane.showMessageDialog(
        null,
        message,
        "WARNING!",
        JOptionPane.INFORMATION_MESSAGE
    );
}

private void showCantDoThat() {
    JOptionPane.showMessageDialog(
        null,
        "Karel can't do that",
        "WARNING!",
        JOptionPane.INFORMATION_MESSAGE
    );
}
```

Para Canvas del paquete shapes se tienen las siguientes librerías

```
import javax.swing.*;
import java.awt.*;
import java.util.List;
import java.util.*;
```

De parte de las figuras se tiene la librería *java.awt* todas sus clases, en *Circle* se importa esa librería y todas las clases de *java.awt.geom*.

Estas son las referencias de extensión que se deben de tener para extender este proyecto.

C BONO. Nuevos requisitos funcionales. [RobotMaze](#). [En [lab01.doc](#)]

Extiendan la mini-aplicación [RobotMaze](#) con los siguientes requisitos:

- Hacer un buen movimiento (la máquina decide). Explique la estrategia. Deshacer el último movimiento.
 - Deshacer se considera un movimiento.
1. Diseñen, es decir, definan los métodos que debe ofrecer.
 2. Implementen los nuevos métodos. Al final de cada método realicen una prueba indicando su propósito. Capturen las pantallas relevantes.

RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)

Gualdrón 16 horas - Villagrán 18 horas

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?

Aceptable pero no destacable ya que debemos mejorar en la comunicación y como establecemos ideas antes de la programación y los patrones de esta.

3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?

Si, la de estar constantemente en comunicación y la programación a pares fue la más útil ya que nos obligaba a estar atento y actualizado del avance de nuestro compañero.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Nuestro mayor logro fue coordinar el laberinto con las colisiones ya que fue complicado elaborar la lógica de las colisiones y la generación del laberinto.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor problema técnico fueron las colisiones, para resolverlo, vimos videos de como generaban soluciones para este problema y pedimos ideas a la IA.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

La comunicación constante fue un gran punto a favor, pero debemos mejorar la metodología de trabajo individual.

7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

Utilizamos más que todo la IA para comprender problemas y para utilizar nuestro repositorio, por tanto, decidimos que la IA fue la referencia más útil.

Aquí se tienen las citas de las inteligencias artificiales y los videos de YouTube, presentadas en un formato estándar (estilo APA):

Inteligencias Artificiales:

- **Gemini:** Google. (2023). *Gemini* (Modelo de lenguaje grande). Google. <https://gemini.google.com/>
- **ChatGPT:** OpenAI. (2022). *ChatGPT* (Modelo de lenguaje grande). OpenAI. <https://chat.openai.com/>

Videos de YouTube:

- **Video 1 (RyiSnow):** RyiSnow. (2021, 1 de noviembre). *Collision Detection - How to Make a 2D Game in Java #6* [Archivo de Video]. YouTube. <https://youtu.be/oPzPpUcDiYY>
- **Video 2 (Bro Code):** Bro Code. (2020, 11 de julio). *Java collision detection* ✨ [Archivo de Video]. YouTube. <https://youtu.be/qlr2XYZrznI>

Se usó la herramienta GitHub para el trabajo en equipo, esto se observa en el siguiente enlace

<https://github.com/SamuelVillagran/DOPOG06/tree/main/Laboratorios>